

Winter in Data Science (WiDS) 2025

Intro to Reinforcement Learning: Weekly Assignments and Project Report

Daksh Maahor

February 1, 2026

Abstract

This report summarizes my work for the *Winter in Data Science (WiDS) 2025 – Intro to Reinforcement Learning* program. Over four weeks, I worked through a series of assignments that introduced key ideas in reinforcement learning (RL): multi-armed bandits, Markov decision processes (MDPs), value functions and Bellman optimality equations, and dynamic programming methods such as policy iteration and value iteration. The coursework culminated in a final project where I implemented a dynamic-programming-based solver for the 15-puzzle, combining approximate value iteration over masked state spaces with a fallback A* search. In this report I describe what I implemented, what I learned conceptually, and how the ideas connect across the weeks.

Contents

1	Introduction	3
2	Background and Preparatory Work (Week 0)	3
2.1	Programming and Visualization Foundations	3
2.2	Key Takeaways from Week 0	3
3	Multi-Armed Bandits (Week 1)	4
3.1	Problem Setup	4
3.2	Algorithms Implemented	4
3.3	Experimental Setup and Results	4
3.4	Lessons from Week 1	5
4	MDPs and Bellman Optimality (Week 2)	6
4.1	Markov Decision Processes	6
4.2	Value Functions and Bellman Equations	6
4.3	Key Takeaways from Week 2	6
5	Dynamic Programming for Control (Week 3)	7
5.1	Policy Iteration: Jack's Car Rental	7
5.2	Value Iteration: Gambler's Problem	7
5.3	Solving Lights Out with DP	7
5.4	Lessons from Week 3	8
6	Final Project: 15-Puzzle Solver with Dynamic Programming and A*	8
6.1	Problem Description	8
6.2	State Representation and Utilities	8
6.3	Stage-Wise Decomposition and Masked States	9
6.4	Value Iteration Solver	9

6.5	Planning and A* Fallback	9
6.6	Experiments and Observations	10
7	Reflections and Future Work	10

1 Introduction

Reinforcement learning (RL) is a framework for sequential decision making under uncertainty, where an agent interacts with an environment to maximize cumulative reward [1]. Unlike supervised learning, where one is given labelled examples, RL emphasises *trial-and-error learning* and the trade-off between *exploration* and *exploitation*.

This report documents my learning journey through the WiDS 2025 Intro to RL program. The structure of the program, as reflected in my repository, can be summarized as:

- **Week 0:** Python, NumPy, and Matplotlib warm-up notebooks.
- **Week 1:** Multi-armed bandit (MAB) problems and basic exploration strategies.
- **Week 2:** Markov decision processes (MDPs) and Bellman optimality equations.
- **Week 3:** Dynamic programming methods — policy iteration, value iteration, and applications to benchmark RL problems (Jack’s Car Rental, Gambler’s Problem, Lights Out).
- **Project:** A 15-puzzle solver (‘fifteen.dp.py’) using value iteration on masked state spaces combined with an A* search fallback.

The rest of this report is organized by topic rather than strictly by calendar week. I first discuss the preparatory material, then each assignment, and finally the project, with reflections at the end.

2 Background and Preparatory Work (Week 0)

2.1 Programming and Visualization Foundations

Before starting RL concepts, Week 0 focused on ensuring familiarity with the core tools used in the assignments:

- **Python basics** (‘python-tutorial.ipynb’): control flow, functions, basic data structures, and idiomatic Python code.
- **NumPy** (‘numpy-tutorial.ipynb’): vectorized operations, broadcasting, generating random variables, and manipulating arrays. These skills are particularly important for implementing RL algorithms efficiently, since state and value functions are often represented as arrays.
- **Matplotlib** (‘matplotlib-tutorial.ipynb’): plotting learning curves, reward trajectories, and value functions, which is essential for diagnosing the behaviour of RL algorithms.

2.2 Key Takeaways from Week 0

From this preparatory work, the most important outcomes for me were:

- Comfort with **vectorized thinking**: writing code where operations happen over arrays at once, instead of explicit Python loops.
- The ability to produce **clear visualizations** of results, which I later used to plot bandit performance, value functions and policies.
- A smoother transition into mathematical RL code, since NumPy allows a close match between the equations in the textbook and their implementation.

3 Multi-Armed Bandits (Week 1)

3.1 Problem Setup

The Week 1 assignment, described in ‘Week 1/README.md’, introduced the *multi-armed bandit* problem. In this setting, an agent repeatedly chooses among k (in this case $k = 10$) actions (“arms”), each providing stochastic rewards with unknown means. The goal is to maximize the expected sum of rewards over time.

The key conceptual challenge is the **exploration-exploitation trade-off**:

- **Exploration**: trying arms whose value is uncertain in order to learn more about them.
- **Exploitation**: choosing the arm currently believed to be best to collect high immediate reward.

3.2 Algorithms Implemented

Following Chapter 2 of Sutton & Barto [1], I implemented and experimented with several bandit strategies in the notebook `work_on_bandits.ipynb`. In particular:

- **ϵ -greedy**: with probability $1 - \epsilon$ choose the estimated best arm, and with probability ϵ pick a random arm.
- **Optimistic initial values**: initializing action-value estimates to high values to encourage early exploration.
- **Upper-Confidence Bound (UCB)**: using an optimism-in-the-face-of-uncertainty principle to balance exploration and exploitation.

3.3 Experimental Setup and Results

I simulated a k -armed bandit with $k = 10$ arms, where each arm’s true mean reward was drawn from a normal distribution. For each algorithm:

- I ran multiple independent runs (e.g., 2000 runs, 1000 time steps each).
- I recorded the **average reward** per time step.
- I also plotted the **percentage of times the optimal action** was chosen over time.

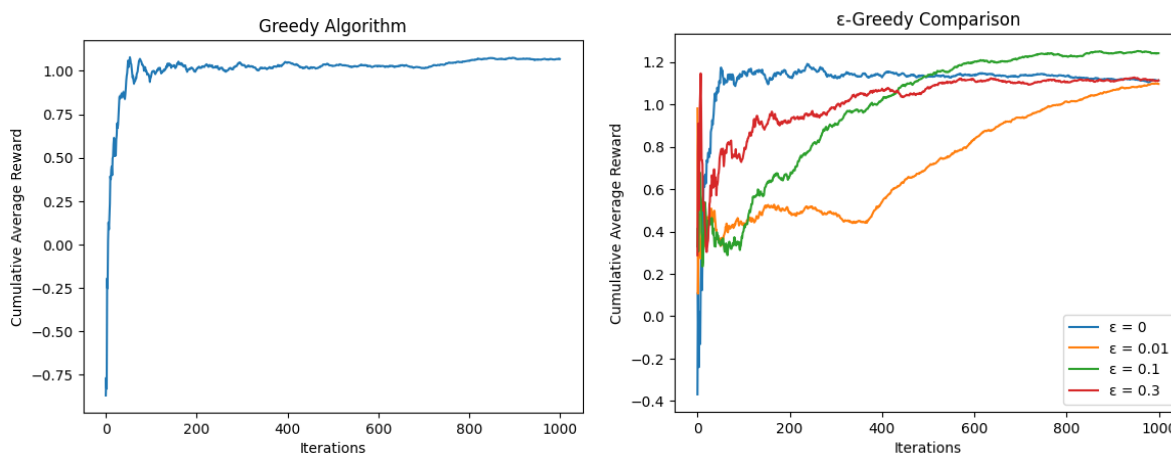


Figure 1: Average reward over time for greedy and ϵ -greedy algorithms.

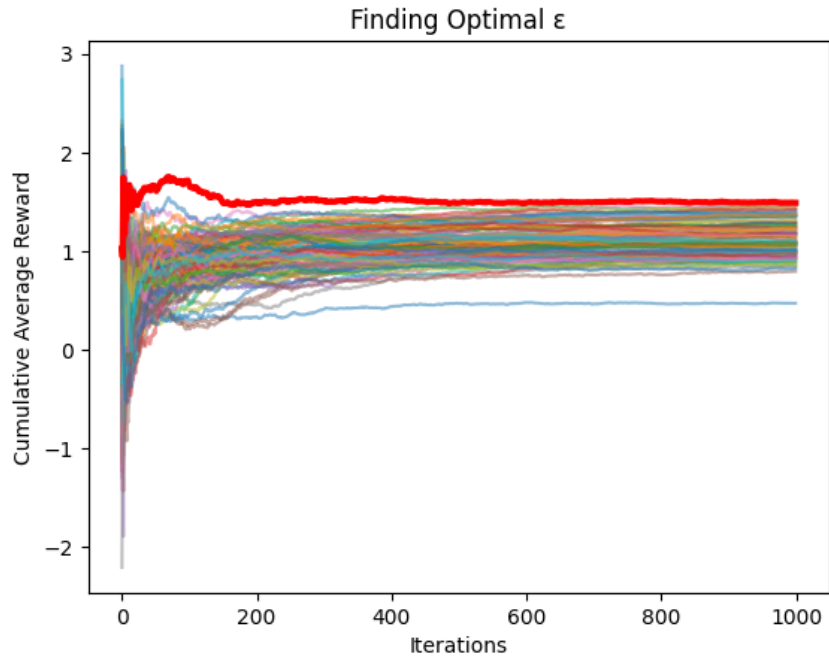


Figure 2: Finding the optimal ε for ε -greedy algorithm.

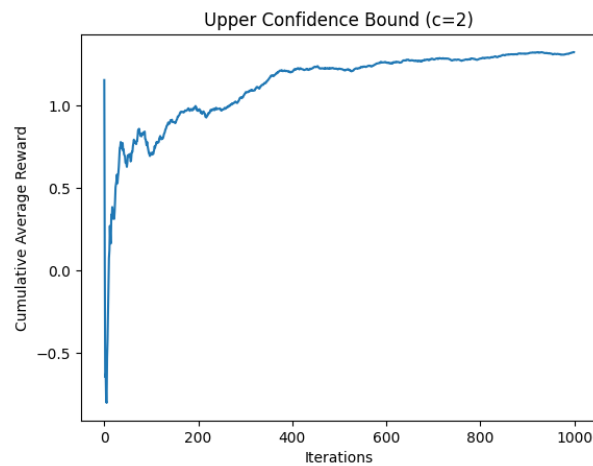


Figure 3: Average reward over time for Upper-Confidence Bound (UCB) algorithm.

3.4 Lessons from Week 1

The main things I learned from this week were:

- Simple strategies like ε -greedy can perform surprisingly well if ε is tuned appropriately.
- Exploration strategies have a visible impact on both the speed of learning and the final performance.
- This bandit setting, though simpler than full RL, already captures the core tension between exploring and exploiting.

4 MDPs and Bellman Optimality (Week 2)

4.1 Markov Decision Processes

Week 2 focused on **Markov decision processes (MDPs)**, as described in Grokking Deep RL (Chapter 2) and Sutton & Barto (Chapter 3). In an MDP, at each discrete time step t :

- The environment is in some state S_t .
- The agent chooses an action A_t .
- The environment provides a reward R_{t+1} and transitions to a new state S_{t+1} .

The key assumption is the *Markov property*: the next-state distribution depends only on the current state and action, not on the full history.

In the ‘Assignment.2.ipynb’ notebook, I modelled different environments as MDPs, including a *Lights Out* puzzle. The process involved:

- Defining the **state space** (e.g., board configurations).
- Defining the **action space** (e.g., pressing a particular light).
- Specifying the **transition dynamics** (how pressing a light changes the board).
- Choosing a **reward structure** (e.g., -1 per move until the board is solved).

4.2 Value Functions and Bellman Equations

Week 2 also introduced value functions and the Bellman optimality equations. For a given policy π , the state-value function is

$$v_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s],$$

and the optimal value function satisfies the *Bellman optimality equation*:

$$v_*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')].$$

Through the assignments, I practiced:

- Writing down Bellman equations for small MDPs.
- Interpreting them in terms of backup diagrams and recursive relationships.
- Connecting the mathematical definitions to the code that computes value functions.

4.3 Key Takeaways from Week 2

- Most RL problems can be viewed as control problems in an underlying MDP, even when we do not explicitly know its dynamics.
- Value functions encode the *long-term* desirability of states, beyond immediate rewards.
- The Bellman equations form the basis of many RL algorithms, both planning (DP) and learning (TD, Q-learning).

5 Dynamic Programming for Control (Week 3)

Week 3, as described in ‘Week 3/README.md’, focused on dynamic programming (DP) methods for solving small MDPs exactly when a model is available.

5.1 Policy Iteration: Jack’s Car Rental

In Part 1, I implemented **policy iteration** for the Jack’s Car Rental problem (Example 4.2 in Sutton & Barto). The environment describes two locations where cars can be rented and returned according to Poisson processes. The agent decides how many cars to move between the two locations overnight.

The policy iteration algorithm alternates between:

1. **Policy evaluation:** computing v_π for the current policy π .
2. **Policy improvement:** updating π greedily with respect to v_π .

In my notebook `Jack’s Car Rental.ipynb`, I:

- Represented states as pairs (n_1, n_2) of cars at each location.
- Implemented the expected returns under the Poisson rental and return model.
- Visualised the learned policy as a 2D heatmap, similar to Figure 4.2 in the book.

5.2 Value Iteration: Gambler’s Problem

In Part 2, I implemented **value iteration** for the Gambler’s Problem (Example 4.3). The gambler’s state is her current capital $s \in \{0, 1, \dots, 100\}$; at each step she bets some portion of her capital and wins or loses based on a biased coin flip.

Value iteration repeatedly applies the Bellman optimality backup:

$$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')],$$

until convergence.

In `Gambler’s Problem.ipynb` I:

- Implemented the Bellman optimality update for each stake a .
- Monitored the maximum change $\max_s |v_{k+1}(s) - v_k(s)|$ to detect convergence.
- Plotted the resulting optimal policy as a bar graph, matching the characteristic spikes described in the textbook.

5.3 Solving Lights Out with DP

In Part 3, I revisited the Lights Out MDP modelled in Week 2 and used dynamic programming (either policy iteration or value iteration) to find an optimal policy.

Key aspects:

- The state space consists of all possible light configurations (e.g., 2^{n^2} for an $n \times n$ board).
- Actions correspond to pressing specific lights.
- The goal is to reach the all-off configuration in as few moves as possible.

By solving this MDP, I was able to reason about:

- The minimal number of moves required under an optimal policy.
- How DP scales with the size of the board and the state space.

5.4 Lessons from Week 3

- Dynamic programming provides exact solutions when we know the model and the state space is not too large.
- Policy iteration and value iteration are closely related: both rely on Bellman backups but differ in how they interleave evaluation and improvement.
- Visualizing policies (heatmaps, bar plots) is extremely helpful to understand qualitative behaviour.

6 Final Project: 15-Puzzle Solver with Dynamic Programming and A*

The final project, implemented in `Project/fifteen_dp.py`, applies ideas from dynamic programming and heuristic search to the classic 15-puzzle.

6.1 Problem Description

The 15-puzzle consists of a 4×4 board containing 15 numbered tiles and one empty space. A legal move slides a tile adjacent to the empty cell into the empty space. The objective is to transform a scrambled board into the goal configuration

$$(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, \text{empty}).$$

Naïvely, the state space is enormous ($16!$ states), so exact DP over the full space is intractable. My approach follows an idea due to Amin Shali: solve the puzzle *row by row* with *masked state spaces*, then fall back to A* if the DP-derived policy stalls.

6.2 State Representation and Utilities

In the code, a board configuration is represented as a tuple of length 16:

$$\text{Position} = (t_0, t_1, \dots, t_{15}),$$

where each $t_i \in \{0, \dots, 16\}$ and the constant `EMPTY` represents the blank tile. Several utility functions support this representation:

- `encode_state` and `decode_state` pack/unpack a position into a single integer using 5 bits per cell.
- `index_to_rc` and `rc_to_index` convert between linear indices and (row, col) coordinates.
- `legal_moves` and `legal_move_indices` compute which tiles/indices can move into the empty cell.
- `apply_move` and `apply_move_index` apply these legal moves to generate successor positions.

The heuristic function

$$\text{manhattan.distance}(\text{pos})$$

computes the sum of Manhattan distances of all tiles from their goal positions, which is a standard admissible heuristic for the 15-puzzle.

6.3 Stage-Wise Decomposition and Masked States

To make DP tractable, I divided the problem into three stages, represented by the `StageSpec` dataclass:

- **Stage “row1”**: ensure tiles 1–4 are in the correct positions in the first row.
- **Stage “row2”**: ensure tiles 1–8 (first two rows) are correctly placed.
- **Stage “last”**: solve the remaining tiles 9–15.

Each stage keeps track only of a subset of tiles (plus the empty) via a *masked* state:

- `mask_position(pos, keep_tiles)` replaces tiles not in `keep_tiles` by 0, effectively projecting the full board into a smaller abstract state space.

The `terminal` method on `StageSpec` checks when a stage is complete by looking at the *unmasked* position. For example, for `row1`, it checks whether the first row equals (1, 2, 3, 4).

6.4 Value Iteration Solver

The class `ValueIterationSolver` implements value iteration over these masked state spaces:

- It stores a value function V and a greedy policy π per stage, keyed by the integer hash of masked states.
- The method `_generate_reachable_states` performs a backward breadth-first search from sampled terminal states for a stage, limiting the number of stored states via a hard cap (`max_states_per_stage`).
- `_value_iteration_stage` performs the standard value iteration updates with a constant step cost (`move_cost`), stopping when the maximum change δ falls below a threshold `theta`.
- After convergence, it extracts a greedy policy π by choosing, in each state, the action (tile index) that maximizes $r + \gamma V(s')$.

Training all three stages is done by `train_all_stages`, which loops over the stage specifications in order.

6.5 Planning and A* Fallback

To solve a given scrambled board:

- `solve(start)` steps through the stages sequentially, using the greedy policy induced by the masked value function at each stage.
- If all stages are completed and the puzzle is solved, the method returns the sequence of tile moves that achieves the solution.
- If the DP-derived policy stalls or fails to solve within a maximum number of steps, the solver falls back to `astar_solve`, which runs an A* search with the Manhattan heuristic from the current state.

The A* implementation:

- Maintains a priority queue of $(f, g, \text{state_code}, \text{Position})$, where $f = g + h$ and h is the Manhattan distance.
- Tracks best-known g -values and parent pointers to reconstruct the final path of moves.

6.6 Experiments and Observations

In `main()`, the program:

- Starts from the canonical solved configuration.
- Calls `scramble` to perform a fixed number of random legal moves to generate a scrambled but solvable board.
- Trains the value-iteration-based policies for all stages.
- Uses `solve` to plan a sequence of moves and prints a trace of intermediate boards.

Qualitatively, I observed that:

- The staged DP policy often solves the early rows reliably and quickly.
- The approximate nature of the sampled terminal states and masked DP sometimes leads to suboptimal or incomplete policies, motivating the need for the A* fallback.
- Combining approximate DP with exact search is a practical strategy: DP provides a good “prior” policy that reduces the search space that A* must explore.

7 Reflections and Future Work

Across the four weeks and the final project, my understanding of RL evolved from simple bandit problems to full MDPs and planning algorithms. Some overarching reflections:

- **Unifying perspective:** bandits, MDPs, and puzzles like the 15-puzzle can all be seen as instances of agents making sequential decisions under uncertainty.
- **Trade-offs:** exact DP methods are powerful but limited to small or structured state spaces; approximate methods and heuristics are necessary in larger domains.
- **Implementation practice:** working through the equations and implementing them in code (NumPy, Python) greatly deepened my intuition compared to just reading the textbook.

If I had more time, natural extensions would include:

- Implementing model-free methods (e.g., Q-learning, SARSA) on some of the Week 3 MDPs and comparing with DP.
- Exploring deep RL variants (e.g., DQN) on higher-dimensional versions of these problems.
- Improving the 15-puzzle solver with better heuristics or learning-based components.

Acknowledgements

I would like to thank the WiDS 2025 organisers and mentors for providing the reading materials, detailed assignments, and guidance.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [2] M. Morales, *Grokking Deep Reinforcement Learning*, Manning Publications, 2020.